

1 计图挑战热身赛

1.1 赛题说明

本赛道将在数字图片数据集 MNIST 上训练 Conditional GAN (Conditional generative adversarial nets) 模型, 通过输入一个随机向量 z 和额外的辅助信息 y (如类别标签), 生成特定数字的图像。

1.2 框架与代码补充

本次实验工程量非常小, 在原始框架中只需要补充 5 行代码即可完善整个条件对抗生成网络。根据提示, 我们需要在 Discriminator 添加最后一个线性层, 将其输出为一个实数:

```
1 # TODO: 添加最后一个线性层, 最终输出为一个实数
2 nn.Linear(512, 1),
```

接着根据第二个 TODO, 将 d_in 输入到模型中并返回计算结果:

```
1 def execute(self, img, labels):
2     d_in = jt.contrib.concat((img.view((img.shape[0], (- 1))),
3     ↪ self.label_embedding(labels)), dim=1)
4     # TODO: 将  $d\_in$  输入到模型中并返回计算结果
5     return self.model(d_in)
```

第三四个 TODO 都在训练判别器的部分, 根据 TODO 提示和函数调用方法, 去计算真实类别和虚假类别的损失:

```
1 # 损失函数: 平方误差
2 # 调用方法: adversarial_loss(网络输出 A, 分类标签 B)
3 # 计算结果:  $(A-B)^2$ 
4 validity_real = discriminator(real_imgs, labels)
5 #  $d\_real\_loss = adversarial\_loss("""TODO: 计算真实类别的损失函数""")$ 
6 d_real_loss = adversarial_loss(validity_real, valid)
7
8 validity_fake = discriminator(gen_imgs.stop_grad(), gen_labels)
9 #  $d\_fake\_loss = adversarial\_loss("""TODO: 计算虚假类别的损失函数""")$ 
10 d_fake_loss = adversarial_loss(validity_fake, fake)
```

随后在最后一个 TODO 的位置我们补充上我们的学号, 注意是字符串类型的:

```
1 number = "2312236" # TODO: 写入比赛页面中指定的数字序列 (字符串类型)
```

1.3 训练结果

接着就是我们的训练过程，在训练的时候我们记录每轮的 D Loss 和 G loss 并绘制训练损失曲线：

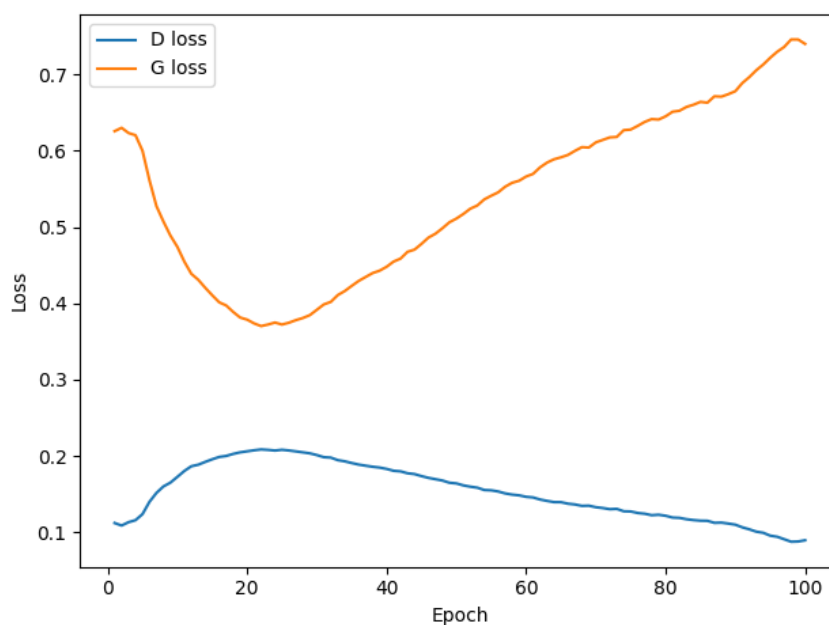


Figure 1: 100epoch 训练损失曲线

可以观察到：1) D_loss 一开始比较低，然后在前 20 个 epoch 左右上升到一个小峰值，之后持续下降，最后接近 0.1。2) G_loss 一开始下降，在大约 20~25 个 epoch 达到最低点，然后一路上升，越训越高。

在后面 D loss 持续下降：判别器越来越自信，越来越容易区分真假；G loss 持续上升：生成器生成的样本越来越骗不过判别器。对抗生成网络的平衡被打破。

然后观察我们生成的手写体图片：



Figure 2: 100Epoch_result

如果我们进一步增加训练轮数，我们就会发现，这种差距进一步被拉大，而生成的图片效果也会降低：

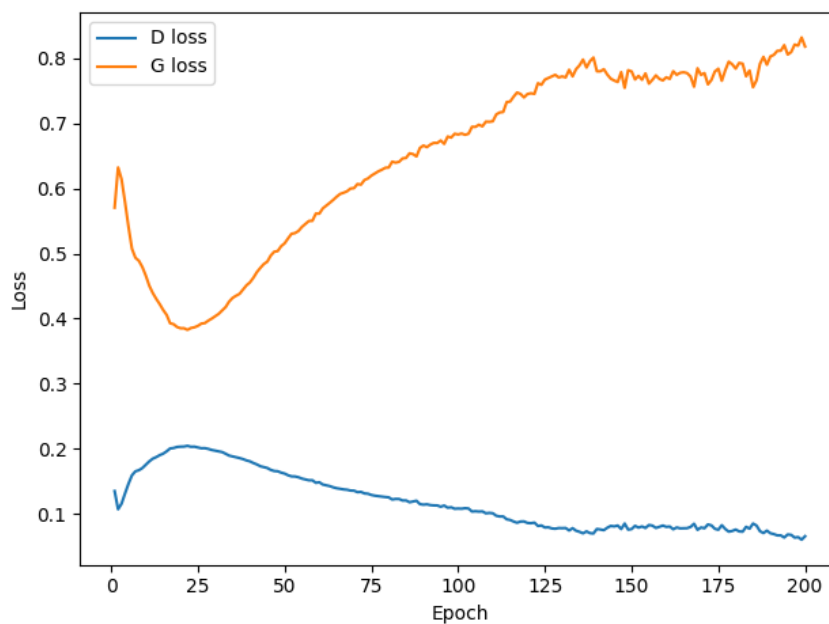


Figure 3: 200epoch 训练损失曲线

以及对应的手写体图片：



Figure 4: Caption

1.4 开源地址

点击[这里](#)，下面是对应的主页截图：

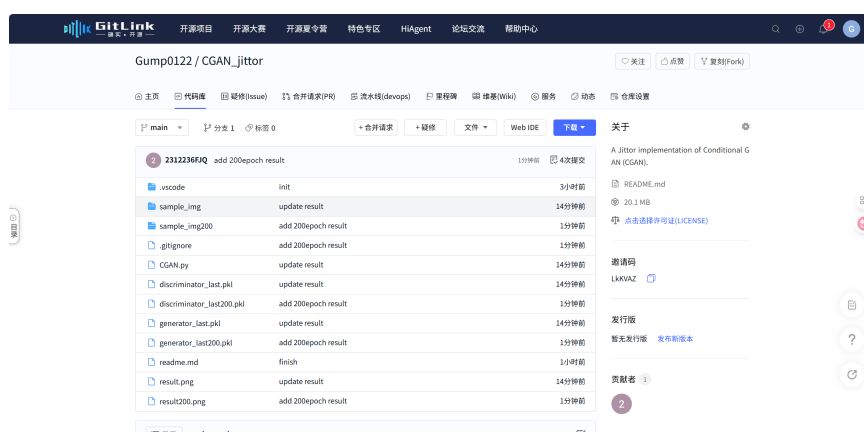


Figure 5: 开源截图